4.27 Cache——替换算法（习题课一）

考纲定位：计算机组成原理 · 第 4 章 存储器（4.27） | 重要度 ★★★★ | 常见考法：选择题/大题（FIFO/LRU 替换过程、命中率、替换次数）

一句话核心：替换题先画 Cache 行状态表，按访问序列逐步更新；FIFO 看调入顺序，LRU 看最近使用顺序

图旁说明：题图与解析图保留；文字只给“考什么 / 常见错因 / 解题模板”，不转写图中解答

一、替换算法总结



提炼

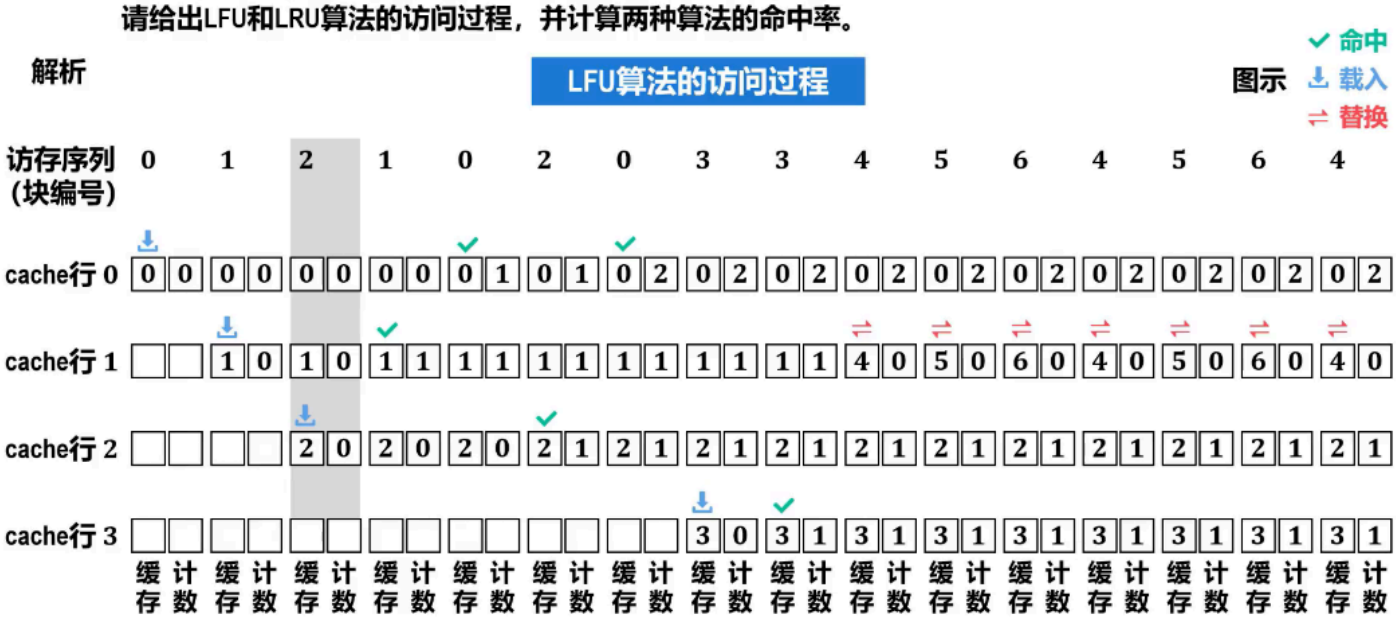
- 直接映射：固定行替换，无需算法
- 全相联：全局选择
- 组相联：组内选择
- FIFO/LFU/LRU/随机是常见策略

二、例题

2.1 例题一

【例题】假设cache有4行、内容初始为空、采用全相联映射方式，按以下编号序列访问相应的主存数据块：
0，1，2，1，0，2，0，3，3，4，5，6，4，5，6，4。
请给出LFU和LRU算法的访问过程，并计算两种算法的命中率。

解析



分析



颠簸（pingpong）
抖动（thrashing）

- 受到早期高频访问数据的影响，这些数据可能在初始化加载时被频繁访问，但之后很少或不再被访问。由于它们起初的访问次数很高，这些数据可能会长时间占用cache空间，导致cache污染。
- 新加入cache的数据由于访问次数较低，很容易被快速替换，尤其是当cache空间有限时。这可能导致cache末端发生“抖动”，即新数据频繁地被加载和淘汰。
- 如果数据访问模式随时间变化，LFU算法可能无法及时适应这种变化，导致一些不再频繁访问的数据仍然保留在cache中，而一些近期变得频繁访问的数据却未能及时加载到cache中。

造成“抖动”的原因？



考什么：按访问序列模拟替换过程 **常见错因**

- 初始 Cache 为空时误判命中
- FIFO 与 LRU 的替换对象混淆
- 命中后忘记更新“最近使用”顺序

解题模板

1. 画表：行号、当前块、FIFO 顺序/LRU 顺序
2. 逐个访问：先判断命中/缺失
3. 命中：更新 LRU；FIFO 不变
4. 缺失且未满：放空行
5. 缺失且已满：按算法选择替换
6. 统计命中/缺失次数

2.2 例题二

【例题】假设cache有4行、内容初始为空、采用全相联映射方式， 按以下编号序列访问相应的主存数据块：
0，1，2，1，0，2，0，3，3，4，5，6，4，5，6，4。
请给出LFU和LRU算法的访问过程，并计算两种算法的命中率。

解析

LRU算法的访问过程

图示 ✓ 命中 ↓ 载入 ⇌ 替换

访存序列（块编号）	0	1	2	1	0	2	0	3	3	4	5	6	4	5	6	4
cache行 0	0	0	0	1	0	2	0	3	0	0	0	1	0	0	0	1	0	2	0	3	0	4	6	0	6	1	6	2	6	0	6	1
cache行 1			1	0	1	1	1	0	1	1	1	2	1	3	1	4	1	5	4	0	4	1	4	2	4	0	4	1	4	2	4	0
cache行 2					2	0	2	1	2	2	2	0	2	1	2	2	2	3	2	4	5	0	5	1	5	2	5	0	5	1	5	2
cache行 3															3	0	3	0	3	1	3	2	3	3	3	4	3	5	3	6	3	7
	缓存	计数	缓存	计数	缓存	计数	缓存	计数	缓存	计数	缓存	计数	缓存	计数	缓存	计数	缓存	计数	缓存	计数	缓存	计数	缓存	计数	缓存	计数	缓存	计数	缓存	计数	缓存	计数

考什么：替换算法对命中率的影响 **常见错因**

- 只算缺失次数，不算总访问次数
- 把命中率写成缺失率
- 忽略初始空 Cache 的 compulsory 缺失

解题模板

1. 总访问次数 N
2. 命中次数 N_c，缺失次数 N_m
3. 命中率 $h = N_c/N$
4. 比较不同算法的 h 与替换次数

知识关联：替换算法（4.26 Cache——替换算法）；直接映射（4.23 Cache——地址映射（直接映射））；组相联（4.25B Cache——地址映射（组相联映射））；习题课二（4.28 Cache——替换算法（习题课二））